Optimizing Layer Normalization in CUDA

A Progressive Optimization Study Using Nsight Compute

Contents

1 Introduction

Layer Normalization is a method used in neural networks that normalizes the activation distribution within a layer to a standard normal distribution. Modern transformer architectures rely on it to help solve two key problems: the vanishing gradient problem and internal covariate shift (ICS). Together, these problems can cause instability in training and longer training times. Layer Normalization is used heavily during both training and inference, so improvements to its performance can have a meaningful impact on the overall execution time of a neural network. This writeup analyzes progressive optimization approaches for Layer Normalization in CUDA and evaluates their performance using Nsight Compute.

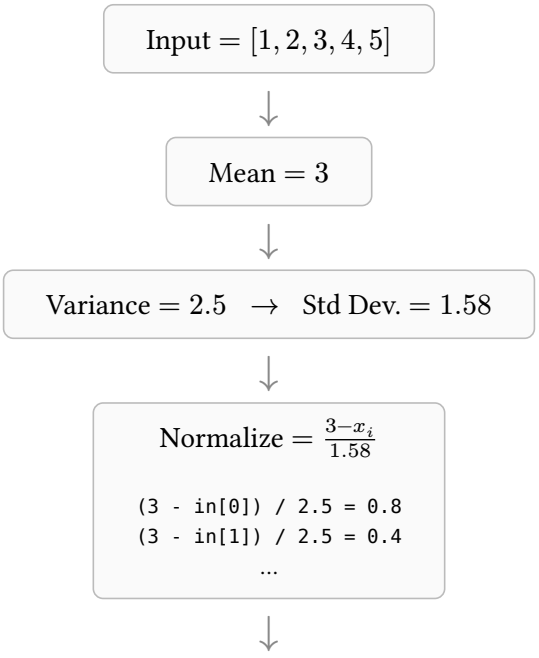
2 Algorithm

The formula may look familiar, as it's an application of the z-score formula to neural networks.

$$\underbrace{\frac{x-\mu}{\sigma}}_{\textit{Z-score}} \quad \longrightarrow \quad \underbrace{\left(\frac{x-\mu}{\sigma+\varepsilon}\right)\gamma+\beta}_{\textit{Layer Normalization}}$$

x	is the input
μ	is the mean
σ	is the standard deviation (derived from the variance)
ε	is the numerical stability term
γ, β	are the learnable parameters

The main calculations in this formula are the mean, variance, and normalization step. These should be calculated in the right order to improve data reuse and reduce memory accesses as much as possible, which is discussed in more detail in the Nsight observations below. Conceptually, Layer Normalization walks through an array of inputs (tokens), finds the mean and variance of each token's vector, and then normalizes the inputs using the Layer Normalization function. An example calculation for one token is shown below.



Output

```
out[0] = 0.8
out[1] = 0.4
...
```

3 Optimizations

3.1 Naive

Idea: One thread computes one token. Each thread loops through the entirety of its token's associated input, calculates the mean and variance, and then normalizes the input.

Why it's slow

- Redundant global memory traffic
- Poor memory locality
- Low data reusability in reductions

Nsight Observations

- Low memory throughput
- Uncoalesced accesses
- Very low SM utilization

3.2 Block Reduction

Idea: One block computes one token, with each thread representing an element in the token embedding.

What Changed?

- Shared memory is used to minimize global memory accesses and increase data reuse
- Parallel reduction is used to calculate the mean and variance

In this implementation, input values are placed into shared memory and reduced there to calculate the mean. Per-element variances are similarly computed and stored in shared memory for reduction. From there, the normalized value is calculated and written to the output array. This approach gives a significant speedup and utilization improvement over the naive kernel.

Nsight Observations

- Warp divergence within the reduction loop
- Extraneous synchronization overhead from operating on shared memory

These observations show that there's still performance to gain by cutting down, or removing, calls to `__syncthreads()`. One way to do this is to use warp reduction instead of operating directly on shared memory.

3.3 Warp Reduction

Idea: One block computes one token. Reduce across the number of warps in the token embedding, then reduce again to get the final mean and variance values.

What changed?

- Reduction using warp-level primitives
- Reduced thread divergence

The one-block-one-token model and the use of shared memory stay the same. The difference is that, per token, the token's embedding is reduced across the number of warps that make it up. Using warp reduction, calls to `__syncthreads()` are reduced and thread divergence within the reduction loop is removed,

allowing the reduction to happen at the warp level within registers before the reduced value is written to shared memory.

Nsight observations
- Low compute throughput utilization

Using warp-level primitives increases speed and throughput at the cost of SM throughput %. This isn't inherently a negative — the tradeoff makes this the fastest approach covered here. The main drawback is in the second warp reduction: for example, a 256-element token embedding produces 256 / 32 = 8 partial results after the first reduction, and the second reduction then uses warp primitives to reduce over only those 8 elements.

3.4 Fused

Idea: One block computes one token, combining warp reduction and shared-memory reduction.

What changed?
- Warp reduction is used for the first reduction pass
- Direct shared-memory reduction is used for the second reduction pass

Nsight observations
- Compute and memory throughput are well balanced
- Further improvements are still possible

Combining warp reduction with direct shared-memory reduction increases SM throughput % over the pure warp-reduction kernel.

> *Open question: not yet confirmed whether the remaining memory accesses in this kernel are actually uncoalesced — worth revisiting with Nsight.*

4 Results

Kernel	Runtime (µs)	Speedup	Memory Throughput (GB/s)	SM Throughput %
Naive	419.36	1.0×	3.23	0.36
Block	24.83	16.9×	46.25	69.17
Warp	11.97	35.0×	93.64	46.33
Fused	15.33	27.4×	75.58	68.78

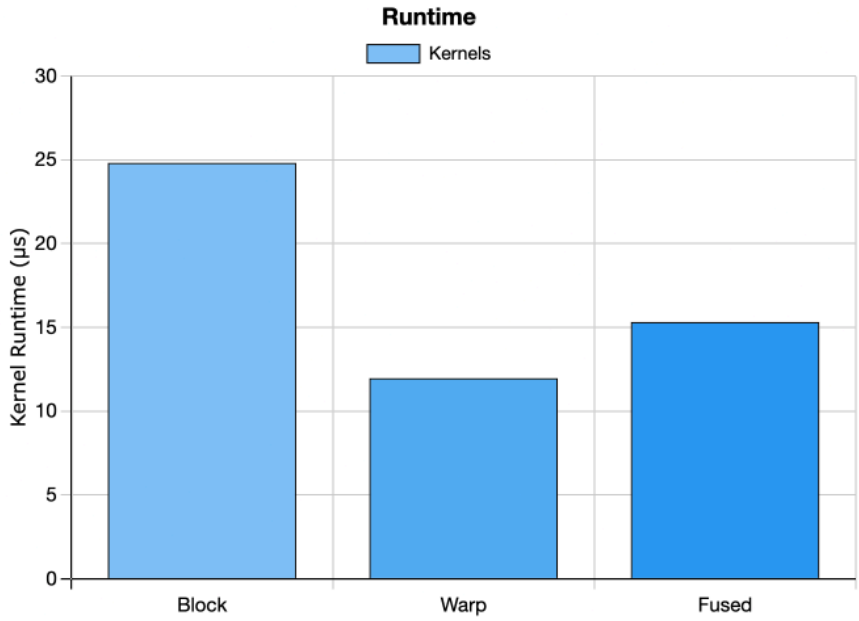


Figure 1: Kernel runtime.

Naive is excluded here — at 419.36 µs it would dwarf the other three

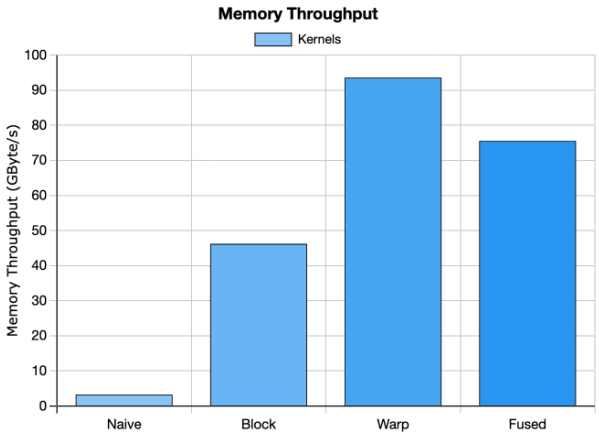


Figure 2: Memory throughput by kernel.

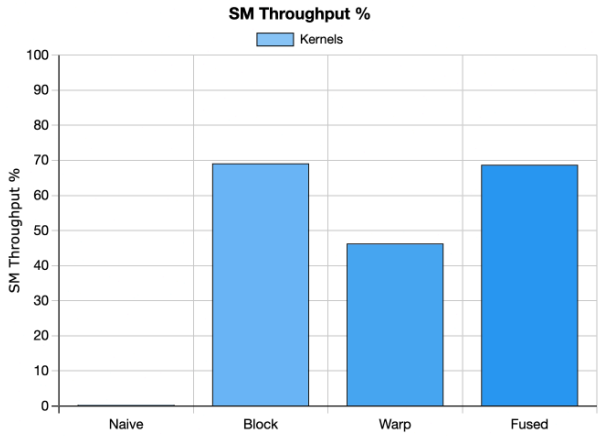


Figure 3: SM throughput % by kernel.

5 Conclusion

Starting from a naive implementation of Layer Normalization, I implemented and measured versions using shared memory, warp-level reduction, and a fused approach. Each optimization improved substantially on the naive kernel: runtime dropped from 419.36 µs to as low as 11.97 µs, alongside much better hardware utilization. Nsight Compute revealed the memory-bound nature of Layer Normalization and helped identify bottlenecks and validate each optimization along the way. Next steps include vectorized loads and applying these same techniques to other essential neural network calculations like GEMM.